

HorRAGor BOT



Introduction & Objectifs.....	1
Architecture.....	2
Système multi agent peer to peer.....	2
Graph.....	3
nodes.py — Les Ouvriers (La Logique Métier).....	3
router.py — Les Aiguilleurs (La Logique Décisionnelle).....	3
pipeline.py — L'Ingénieur Système (Le Câblage).....	4
Monitoring du système multi agent.....	4
Langfuze.....	4
Docker.....	4
Utilisation.....	5

Introduction & Objectifs

L'objectif de ce projet est de faire évoluer **HorRAGor**, notre agent conversationnel spécialisé dans l'univers de l'horreur. Dans la phase précédente, vous avez conçu le "cerveau" de l'application via un agent autonome unique basé sur l'architecture ReAct, capable d'interroger son savoir vectoriel et de chercher sur le web. Cependant, en production, un agent monolithique unique montre vite ses limites : saturation du contexte (prompt drowning), risques de conflits décisionnels et affaiblissement de la plume narrative au milieu des logs techniques de données.

L'objectif de cette **Partie 3** est de briser ce modèle centralisé pour concevoir une architecture **Multi-Agent distribuée et collaborative** sous **LangGraph**. Vous allez diviser les responsabilités de HorRAGor en une équipe de profils ultra-spécialisés qui partagent un état commun (**State**) et qui se passent le relais.

Le système multi-agent HorRAGor devra articuler et faire collaborer trois entités distinctes :

- **L'Agent RAG (Le Chercheur Local)** : Premier point de contact, il interroge intelligemment le savoir structuré et vectoriel pour extraire le lore brut des œuvres d'horreur et corriger les approximations de l'utilisateur.
- **L'Agent Scraper (L'Enquêteur du Web)** : Déclenché via un routage conditionnel uniquement si la base locale est incomplète, sa mission est d'aller creuser le web en direct pour extraire les anecdotes et détails manquants sur une œuvre spécifique.
- **L'Agent de Narration (L'Écrivain Gothique)** : Isolé de toute la plomberie technique pour éviter la collision de tokens et la paresse de génération, il récupère uniquement la synthèse des données brutes pour les emballer dans une atmosphère textuelle terrifiante, immersive et hautement romancée.

À l'issue de ce chapitre, vous maîtriserez la gestion de graphes complexes, le routage dynamique, l'isolation de contexte (*context trimming*) et l'alignement comportemental de LLM locaux en environnement multi-IA.

Architecture

```
horRAGor-project/
├── data/
│   └── faiss_index/           # Index vectoriel local de l'horreur
├── app_frontend.py           # Interface Streamlit
├── src/
│   ├── main.py               # Serveur FastAPI (Uvicorn)
│   ├── config.py             # Configuration des LLM (Ollama) et chemins
│   ├── models/
│   │   └── state.py          # Le State de confiance (Mémoire commune)
│   ├── tools/
│   │   ├── rag_tool.py       # Outil natif de recherche dans FAISS
│   │   └── scraper_tool.py    # Outil natif de recherche Web
│   └── graph/
│       ├── nodes.py          # Logique des nœuds (RAG, Scraper, Narration)
│       ├── router.py          # Fonctions d'aiguillage (Edges conditionnels)
│       └── pipeline.py        # Câblage et compilation du StateGraph
```

Système multi agent peer to peer

Dans l'arborescence et le flux que l'on va définir, il n'y aura **aucun agent "Chef de Projet" central** (pas de nœud manager qui analyse tout et réorganise le travail à chaque tour de table). Les agents travaillent en chaîne direct :

1. L'**Agent RAG** s'allume en premier et fait sa fouille locale dans FAISS.
2. Le **Routeur** (`router.py`) intervient comme une aiguille sur un rail de train : il regarde si la récolte locale est suffisante.
 - Si oui → Il envoie le flux directement sur le bureau de l'**Agent de Narration**.
 - Sinon → Il déroute le flux vers l'**Agent Scraper** pour enrichir le dossier, qui l'enverra ensuite à la Narration.
3. L'**Agent de Narration** ferme la marche en injectant la plume d'horreur.

Graph

Voici le rôle exact et le cahier des charges de chacun de ces composants.

[nodes.py](#) — Les Ouvriers (La Logique Métier)

Ce fichier regroupe l'ensemble des **nœuds** du graphe. Un nœud n'est rien d'autre qu'une fonction Python pure chargée d'exécuter une tâche métier spécialisée.

- **Ce qu'il contient** : Les fonctions associées à vos trois profils d'IA (`rag_node`, `scraper_node`, `narration_node`).
- **La règle d'or LangGraph** : Chaque fonction prend obligatoirement l'état actuel (`state: AgentState`) en paramètre unique et retourne un dictionnaire contenant les modifications ou les nouveaux messages à fusionner dans le `State`.
- **Ce qu'on y fait concrètement** : C'est ici que l'on configure le comportement de chaque agent : on récupère les variables globales, on injecte les `SystemMessage` spécifiques (le prompt de l'enquêteur, le prompt de l'écrivain gothique) et on invoque le LLM (avec ou sans outils attachés). *Les nœuds ne savent jamais qui a travaillé avant eux ni qui prendra la suite.*

[router.py](#) — Les Aiguilleurs (La Logique Décisionnelle)

Ce fichier centralise l'intelligence dynamique du réseau, matérialisée par les **arêtes conditionnelles** (conditional edges).

- **Ce qu'il contient** : Les fonctions de routage (ex: `should_scrape_or_narrate`).
- **La règle d'or LangGraph** : Tout comme les nœuds, une fonction de routage examine l'état actuel (`state`), mais sa seule et unique responsabilité est de renvoyer une chaîne de caractères (un `str`) correspondant à la prochaine destination.
- **Ce qu'on y fait concrètement** : On y écrit les règles logiques ou les filtres MLOps (ex: « Est-ce que le message de l'agent RAG contient assez d'informations ? Si oui, renvoie "narration", si non, renvoie "scraper" »). Isoler ces fonctions permet de tester l'intelligence algorithmique de votre application de manière unitaire, sans lancer tout le pipeline.

C'est l'atelier d'assemblage final. C'est ici que le graphe prend vie en liant les ouvriers (`nodes.py`) et les aiguilleurs (`router.py`) autour du cahier des charges commun (`state.py`).

- **Ce qu'il contient** : L'instanciation de la classe `StateGraph`, la configuration des liaisons et la compilation.
- **La règle d'or `LangGraph`** : C'est le point de vérité de la topologie de votre réseau.
- **Ce qu'on y fait concrètement** : On y écrit le déroulé architectural strict via trois méthodes clés :
 1. `workflow.add_node()` : On enregistre les fonctions de `nodes.py` dans le moteur.
 2. `workflow.add_edge()` : On trace les autoroutes directes (ex: du bouton `Start` vers le nœud `RAG`).
 3. `workflow.add_conditional_edges()` : On greffe les aiguilleurs de `router.py` aux carrefours stratégiques.
- **Le livrable** : Ce fichier se termine par `app = workflow.compile()`. C'est cet objet final, propre et prêt à l'emploi, qui sera importé par FastAPI pour propulser votre backend.

Monitoring du système multi agent

Langfuse

Langfuse est un outil de monitoring open-source qui trace, évalue et calcule le coût de chaque étape de vos agents LLM en temps réel.

Docker

```
# 1. Clonez le dépôt officiel de Langfuse
git clone https://github.com/langfuse/langfuse.git

# 2. Allez dans le dossier du projet
cd langfuse

# 3. Lancez l'application avec Docker Compose
docker compose up -d
```

Utilisation

```
import os
from langfuse.callback import CallbackHandler

# Remplacez par les clés générées sur votre interface http://localhost:3000

os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."

# On pointe vers le serveur local
os.environ["LANGFUSE_HOST"] = "http://localhost:3000"

# Le reste du code LangGraph + Ollama reste presque le même
langfuse_handler = CallbackHandler()

# ...

# sauf qu'il faut le donner à Lang Graph
# 1. créez la configuration avec le callback Langfuse
config = {"callbacks": [langfuse_handler]}

# 2. passez la config au moment d'appeler l'agent
inputs = {"messages": [("user", "Ta question ici")]}
response = agent.invoke(inputs, config=config)
```

Etape

Allez sur le port 3000 puis créez un compte (c'est local donc un faux mail fonctionne). Créez un nouveau projet pour obtenir les clés pour le code.

Lancez le code puis retournez sur langfuse et analysez la consommation de token de votre agent ainsi que la latence exacte de chaque nœud, le tracé complet des appels d'outils, et l'arborescence de décisions de votre graphe.